# Lab: Shell Scripting and Data Wrangling

Operating Systems Laboratory

## Overview

Many tasks performed on computers are repetitive, tedious, or menial. Shell scripting allows users to automate these tasks using the command line. In this lab, students will learn:

- Writing and executing shell scripts
- Using variables, conditionals, loops, and functions in Bash
- Applying data-wrangling tools like `grep`, `awk`, `sed`, `sort`, and `uniq`
- Understanding regular expressions for text pattern matching
- Automating common file and text-processing tasks

# 1 Shell Scripting

## 1.1 Introduction to Shell Scripting

A shell script is a text file containing commands executed sequentially by the shell. Scripts can automate file processing, log analysis, system administration, or repetitive tasks.

Bash (Bourne Again SHell) is a full-featured scripting language that interprets shell commands and allows control structures, functions, and variables.

## 1.2 The Shebang

Every script typically starts with a shebang line:

```
#!/bin/bash
```

This tells the system which interpreter to use. If you run:

```
./myscript.sh
```

the shebang ensures `bash` executes the script. If you instead run:

```
bash myscript.sh
```

the shebang is ignored and bash runs the script anyway.

## 1.3 Comments

Anything following `#` is a comment and not executed:

```
# This is a comment
echo "Hello World" # prints greeting
```

## 1.4 Variables

Variables store data:

```
NAME="Alice"      # assign a string
echo $NAME        # prints Alice
```

**Important:** No spaces around `=`. Variables are untyped and treated as strings by default. Arithmetic can be done with `let`:

```
x=5
let y=x+3
echo $y   # prints 8
```

For floating point arithmetic, use `bc`:

```
echo "scale=2; 10/3" | bc
```

## 1.5 Command-Line Arguments

Scripts can accept arguments:

```
#!/bin/bash
echo "First arg: $1"
echo "Second arg: $2"
```

Call with:

```
./script.sh foo bar
```

Outputs:

```
First arg: foo
Second arg: bar
```

## 1.6 Flow Control

### 1.6.1 Conditionals

```
if [ $1 -eq $2 ]; then
    echo "Arguments are equal"
else
    echo "Arguments are not equal"
fi
```

### 1.6.2 Loops

```
n=5
while [ $n -gt 0 ]; do
    echo $n
    let n=n-1
done

for file in *.txt; do
    echo "Found file: $file"
done
```

## 1.7 Functions

```
greet() {
    echo "Hello $1"
}
greet "World"   # prints "Hello World"
```

Functions do not return values; they output to stdout. Parameters are accessed as 1,2, etc.

## 1.8 Testing Expressions

Use `[ ... ]` or `test`:

```
[ 0 -eq 0 ]; echo $?  # 0 (true)
[ 0 -eq 1 ]; echo $?  # 1 (false)
[ -f file.txt ]; echo $? # check if file exists
```

## 1.9 Activity 1: Basic Scripting Practice

1. Write a script that prints your username, current date, and hostname.
2. Write a script that accepts a filename and prints number of lines, words, and characters.

## 1.10 Activity 2: Conditional and Loops Practice

1. Write a script that prints numbers from 1 to 10.
2. Write a script that iterates over all `.txt` files in a directory and prints their names.

## Graded Task 1: Phonebook Script

Write a shell script `phonebook` that supports:

- `./phonebook new <name> <number>` – adds an entry
- `./phonebook list` – displays all entries
- `./phonebook remove <name>` – deletes all entries for that name
- `./phonebook clear` – deletes all entries
- `./phonebook lookup <name>` – prints all numbers for that name

**Example:**

```
$ ./phonebook new Linus 101-110-0111
$ ./phonebook list
Linus 101-110-0111
$ ./phonebook lookup Linus
Linus 101-110-0111
$ ./phonebook remove Linus
$ ./phonebook list
phonebook is empty
```

# 2 Data Wrangling

## 2.1 Overview

Data wrangling, also known as data munging or preprocessing, involves cleaning, restructuring, and transforming raw data into a usable format for analysis. In this lab, students will work with two types of files:

- `data.txt` – a plain text file with lines containing an ID and three columns of data: two numerical values and one text label.
- `logs.log` – a log file containing timestamps, usernames, actions, and IP addresses, simulating real-world server logs.

Students will learn how to extract meaningful information from these files using command-line tools that are commonly used in data processing pipelines.

## 2.2 Tools and Commands

### 2.2.1 grep: Pattern Matching

`grep` is used to search for lines matching a specific pattern. It supports basic and extended regular expressions and is extremely fast for text searches.

```
# Search for lines containing the word FAILED in logs.log
grep "FAILED" logs.log

# Case-insensitive search for "login"
grep -i "login" logs.log

# Use extended regex to match lines starting with 3 digits
grep -E "^[0-9]{3}" data.txt
```

**Options:**

- `-i` : case-insensitive search
- `-v` : invert match (lines that do NOT match)
- `-c` : count matching lines
- `-n` : show line numbers
- `-E` : use extended regular expressions

**Example Use-Case:** Extract all failed login attempts from a log file:

```
grep "FAILED LOGIN" logs.log
```

### 2.2.2 sed: Stream Editor for Substitution and Filtering

`sed` is a powerful stream editor used for transforming text in a file or stream. It supports substitution, deletion, insertion, and more.

```
# Replace all occurrences of 'LOGIN' with 'SIGNIN'
sed 's/LOGIN/SIGNIN/g' logs.log

# Delete empty lines in a file
sed '/^$/d' data.txt

# Print only lines 2 to 5 from data.txt
sed -n '2,5p' data.txt
```

**Options:**

- `s/pattern/replacement/flags` : substitution
- `d` : delete lines
- `-n` : suppress automatic printing (useful with `p`)
- `g` : global substitution in each line

**Example Use-Case:** Standardize action names in log data:

```
sed 's/FILE_ACCESS/ACCESS/g' logs.log > cleaned_logs.log
```

### 2.2.3 awk: Column-Based Processing and Reporting

`awk` is a versatile text-processing language designed to work with structured data in columns. It can filter, transform, compute statistics, and generate reports.

```
# Print the 1st and 3rd columns from data.txt
awk '{print $1, $3}' data.txt

# Print rows where the 3rd column is greater than 50
awk '$3 > 50' data.txt

# Compute the sum of the 3rd column
awk '{sum+=$3} END {print sum}' data.txt
```

**Key Concepts:**

- Columns are accessed as `$1`, `$2`, etc.
- Patterns can filter rows: `$3 > 50` selects rows where column 3 is greater than 50.

- Actions are enclosed in curly braces {}: {`print $1`}
- Built-in variables: `NR` = line number, `NF` = number of fields in the current row.

**Example Use-Case:** Find the average value in the 3rd column:

```
awk '{sum+=$3; count++} END {print sum/count}' data.txt
```

### 2.2.4 sort and uniq: Sorting and Counting Unique Lines

These commands are used together to organize and summarize data.

```
# Count unique usernames in logs.log
awk '{print $3}' logs.log | sort | uniq -c

# Sort numbers in ascending order (column 2 of data.txt)
sort -n -k2 data.txt

# Sort by 2nd column
sort -k2 data.txt
```

**Options:**

- `-n` : numeric sort
- `-r` : reverse sort
- `-k` : specify column for sorting
- `uniq -c` : count duplicates

**Example Use-Case:** Find the top 5 most frequent IP addresses:

```
awk '{print $5}' logs.log | sort | uniq -c | sort -nr | head -5
```

### 2.2.5 cut: Extracting Specific Columns

`cut` extracts selected fields or characters from each line.

```
# Extract the username column from logs.log
awk '{print $3}' logs.log

# Extract the IP address column from logs.log
awk '{print $5}' logs.log
```

**Note:** For space-delimited files like our `logs.log`, `awk` is often more convenient than `cut` since it handles multiple spaces automatically.

## Graded Tasks: Data Wrangling

### Task 1: Log Analysis

Using `logs.log`:

- Count the total number of lines containing "FAILED" (failed actions).
- List all unique usernames present in the file.
- Identify the top 5 IP addresses with the most failures.

### Task 2: Data Summary

Using `data.txt`:

- Extract all rows where the second column is greater than 100.
- Compute the average of the third column for these filtered rows.
- Save the results to `summary.txt`.